

SmartCache: Two-Dimensional KV-Cache Similarity for Efficient Long-Context LLM Decoding

Chen Shen
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Hao Chen
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Kaining Hui
*College of Information Science
and Engineering
Northeastern University
Liaoning, China*

Zhixin Zhao
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Yang Cheng
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Yitao Hu*
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China
yitao@tju.edu.cn*

Sheng Chen
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Xiulong Liu
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Keqiu Li
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Abstract—Large language models (LLMs) achieve state-of-the-art performance in many NLP tasks but incur prohibitive memory-access and compute costs when processing very long contexts due to linearly growing KV Cache. Existing static sparsification methods rely on fixed heuristics, while dynamic schemes incur substantial runtime overhead. To address this trade-off, we propose SmartCache, a sparse inference system that exploits two-dimensional KV Cache similarity across adjacent decoding iterations and neighboring layers. SmartCache combines a similarity-driven dual-path selection algorithm, which adaptively reuses TopK KV entries from both the previous iteration and the preceding layer with a rolling-array cache index manager that reduces index storage complexity from $O(L \cdot k)$ to $O(k)$. We analyze the layer and iterative sparse patterns of KV Cache in long context LLM decoding and show that SmartCache maintains semantic consistency while drastically reducing redundant computation and memory traffic. Extensive experiments on Llama-3-8B-Instruct-Gradient-1048k, Qwen2.5-7B-Instruct-1M, and glm-4-9b-chat-1m across four long-context benchmarks report up to 30.5% end-to-end latency reduction and 15%–23% average latency reduction, with inference accuracy degradation constrained within 2% and occasional slight improvements. These results indicate that SmartCache offers a practical, high-accuracy solution for scalable long-sequence LLM inference.

Index Terms—Cloud Computing; Distributed LLM Inference; Attention Sparsity; KV Cache Similarity;

This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501000, and in part by the National Natural Science Foundation of China under Grant 62202328.

I. INTRODUCTION

The rapid advancement of transformer-based large language models (LLMs), exemplified by flagship architectures such as GPT-4[1], Gemini-Pro-1.5[2], and Claude-3[3], has revolutionized capabilities in natural language understanding and generation. A defining trend in their evolution is the exponential growth of supported context lengths, from early models constrained to 2k tokens to contemporary ones that process sequences exceeding 1M tokens[2, 4–6]. Such scalability enables unprecedented tasks, such as analyzing entire code repositories[7], summarizing multi-chapter scientific literature[8], extra long context QA[9] or maintaining coherent multi-session dialogues[10].

While scaling LLMs has advanced their capabilities, it also introduces significant computational challenges. Autoregressive decoding requires storing and repeatedly accessing a KV cache that grows linearly with context length. For instance, a 200k-token input in Llama-3-8B [11] produces a 25GB KV cache, and reading this cache from HBM to SRAM incurs over 16 milliseconds of latency [12]. In extra long context settings, the KV cache size can surpass the model parameters by more than an order of magnitude, making memory access a major bottleneck for end-to-end decoding.

Recent research has focused on KV cache sparsification techniques to address limitations. Static methods[13–16] pre-determine retention rules, such as reserving fixed window regions or prioritizing tokens via early-layer feature extraction.

While computationally lightweight, such rigid heuristics often fail to adapt to dynamic attention patterns, compromising accuracy in tasks requiring long-range dependency resolution. In contrast, dynamic approaches[17–20] employ runtime token selection mechanisms to better preserve the critical context such as adaptive hashing or blockwise eviction. Yet frequent cache updates impose high overhead, making these methods impractical for latency-critical applications and motivating a sparse inference approach that balances efficiency with accuracy.

In this work we propose SmartCache, a novel inference system that addresses these limitations by leveraging the two dimensional similarity of KV Cache both inter layer and inter iteration to enable efficient high accuracy sparsification. First, SmartCache introduces a similarity-aware sparse selection algorithm that dynamically reuses critical KV Cache from adjacent layers and recent decoding steps. By exploiting the structural similarity observed in long-context tasks, this design drastically reduces redundant computations. Second, SmartCache incorporates a hierarchical cache index manager that minimizes memory overhead through rolling array optimizations, reducing KV Cache index storage complexity from $O(L \cdot n)$ to $O(n)$. These innovations ensure very low loss of inference accuracy, maintain semantic consistency across decoding steps, while significantly reducing selection costs and hence end-to-end inference latency.

The contributions of this paper are threefold:

- Firstly, SmartCache is the first to identify the two-dimensional sparsity feature of KV Cache, and makes a detailed analysis of this two-dimensional sparsity, exploring the potential of exploiting it in long context decoding.
- Second, SmartCache introduces a dual-path selection reuse mechanism with an adaptive recalculation mechanism, which balances spatial context propagation and temporal continuity, and achieves efficient reuse of attentional patterns with little sacrifice in accuracy.
- Third, extensive experiments on three LLMs(Llama-3-8B[11], Qwen2.5-7B[21], GLM-4-9b[22]) and 15 long context tasks in four different benchmarks[7–10] show that SmartCache reduces the inference latency by 15%-23%, with a maximum reduction of 30.5%. Compared to current state-of-the-art sparse decoding inference systems, SmartCache reduces under 1% accuracy with the same and even lower sparsity budget. Compared to the full KV Cache, SmartCache accuracy loss is kept within 2% on average while in many cases it even shows slight improvements.

II. RELATED WORK

The growing context length in autoregressive decoding leads to decreasing computational intensity at each decoding step, exacerbating memory bandwidth bottlenecks under the roofline model[23, 24], which causes severe decoding latency[25]. Prior studies[13–17, 19, 20, 26–28] have demonstrated that selectively retaining critical KV Cache can approximate full attention performance while reducing memory traffic. There

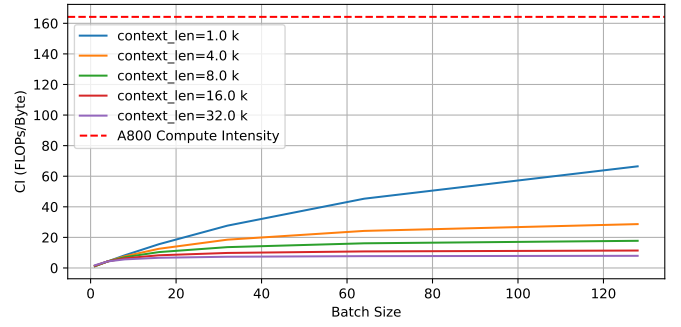


Fig. 1: *CI* trends for the Llama-3-8B model

are two main types of existing systems at the KV Cache selection level: static selection and dynamic selection.

A. Static KV Cache Selection Methods.

Existing static approaches predetermine KV Cache retention rules during prefill. For example, StreamingLLM[13] selects only the KV Cache within the attention sink and local window, and SnapKV[14] takes the *TopK* based on the result of KV Cache pooling. PyramidKV[15] allocates the KV Cache budget according to the shape of the pyramid and GemFilter[16] picks out critical tokens and discards the rest of the KV Cache by using the first few layers of the model. These methods use a fixed heuristic or window strategy to statically select the KV Cache in the decode phase, which may omit critical long-range dependencies in complex long-context tasks, thus reducing the accuracy of inference.

B. Dynamic KV Cache Selection Methods.

Dynamic selection mechanisms address static limitations but introduce new overheads. MagicPig[19] leverages locality-sensitive hashing to identify critical tokens but incurs prohibitive computational costs. Quest[17] adopts block-wise dynamic pruning to reduce latency but risks diluting the importance of the token within blocks, eliminating highly attended tokens. ShadowKV[29] enhances selection accuracy through dynamic matching but introduces latency penalties from low-rank decomposition and cache offloading. These methods highlight the unresolved tension between computational efficiency and accuracy preservation in KV Cache sparsification.

III. BACKGROUND

A. LLM Inference in Long-Context Scenarios

During the autoregressive decoding stage of large language models (LLMs) operating in long-context settings, each token generation is conditioned on both the incrementally expanding sequence of decoded tokens and the full input context, which may span tens of thousands of tokens. Formally, given an input context $C = (x_1, x_2, \dots, x_n)$ and a partially decoded output $Y_{<t} = (y_1, y_2, \dots, y_{t-1})$, the decoder computes the probability distribution for the next token y_t as

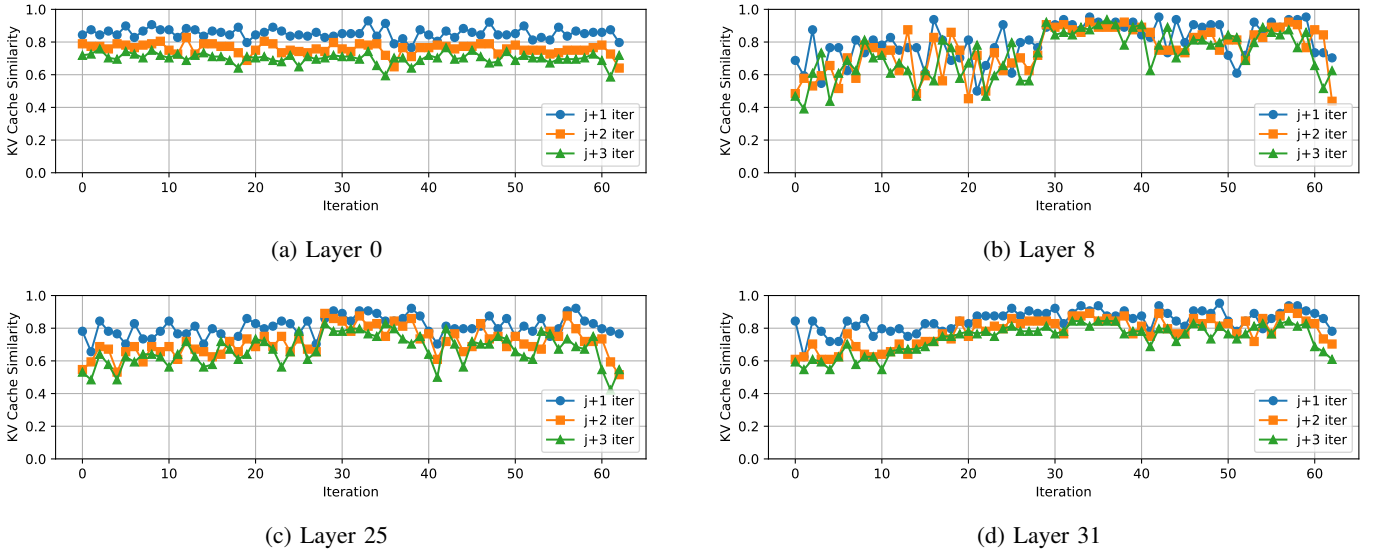


Fig. 2: KV Cache similarity between iteration of 4 different layers of Llama-3-8B-Instruct-1048k model.

$$P(y_t | C, Y_{<t}) = \text{Transformer}(\mathcal{K}_C, Y_{<t}) \quad (1)$$

Here, the Transformer typically consists of a stack of self-attention and feed-forward layers, each of which must attend over the entire concatenated sequence $(C, Y_{<t})$. During each decode step t , the model (1) retrieves KV Cache representations for the static input context C ; (2) computes fresh KV Cache for the newly generated token y_{t-1} ; (3) performs attention over the KV Cache union and new representations; (4) updates the KV Cache with the latest tokens.

Let $\mathcal{K}_C$ and $\mathcal{K}_{Y_{<t}}$ denote the cached KV Cache for the input context and dynamically growing output sequence respectively. While KV Cache reduces redundant computation by reusing historical states, its linear memory growth $\mathcal{O}(n)$ with context length n introduces critical memory bandwidth challenge: Each decoding step requires accessing the full KV cache spanning both C and $Y_{<t}$, resulting in $\mathcal{O}(n)$ memory traffic per token generation. For sequences exceeding $n > 10^4$ tokens, this dominates decode latency due to memory access bottleneck.

B. Sparsity Dynamics in Transformer Decoding

In view of the above considerations, some scholars proposed to utilize the sparsity of KV Cache to alleviate the access bottleneck and reduce the decoding latency[13–15, 17, 26, 28]. KV Cache sparsity refers to the phenomenon that the current query pays much more attention to some important tokens than other tokens during the decoding process, which results in the attention weights of a small portion of tokens are already close to the full amount of results. Thus, selecting a small number of important KV Cache for attention calculation becomes a way to accelerate the decode phase.

However, the sparsity feature exhibited in the Transformer model are not static and are characterized by the following features: (1) deeper layers are more sparse due to semantic

abstraction; (2) KV Cache sparsity is locally correlated across iterations, enabling reuse; (3) sparsity across layers evolves non-monotonically. Efficiently exploiting these patterns to speed up decoding remains an open challenge.

C. Computation and Memory Access Analysis

In order to evaluate bottlenecks in the LLM inference process, a commonly used tool is the roofline model[23]. It identifies performance bottlenecks by analyzing computational intensity and memory bandwidth to provide guidance for inference optimization. For long-context decoding, we adapt this model to characterize the interplay between KV Cache management and attention computation. We define $CI(\text{compute_intensity})$ in (2)

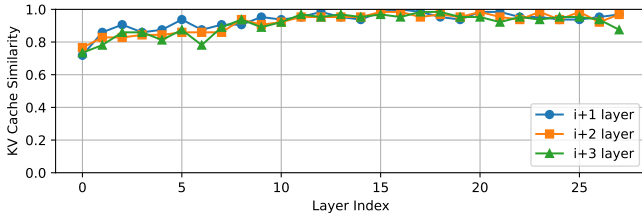
$$CI = \frac{FLOPs}{Memory\ Access} = \frac{\mathcal{F}_{\text{attn}} + \mathcal{F}_{\text{ffn}}}{\mathcal{M}_{\text{attn}} + \mathcal{M}_{\text{ffn}} + \mathcal{M}_{\text{kv_cache}}} \quad (2)$$

Computational intensity (CI) quantifies floating-point operations per memory access unit. High CI indicates compute-bound tasks, while low CI reflects memory-bound constraints. We analyze the CI of Llama-3-8B model in detail, including the computation and accesses of the attention and ffn modules, as well as the KV Cache, the inputs and the accesses of the outputs, to obtain Fig.1:

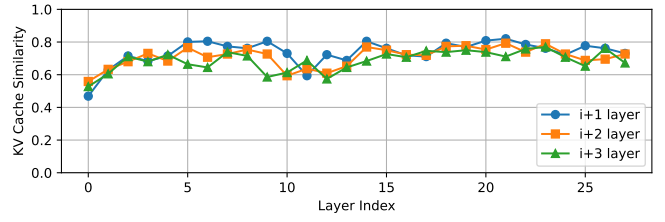
The plot of CI versus $batch_size$ employs colored curves to distinguish sequence lengths and a red dashed line to denote the A800 GPU's CI bound. Based on Fig.1 we conclude that the access bottleneck of the Llama-3-8B model is more severe as the sequence length increases, which gives an opportunity to reduce memory access and thus speed up the inference.

IV. MOTIVATION

Each newly generated token must store its KV Cache for every layer and attention head, causing the KV Cache size and its associated memory traffic to grow linearly with context length



(a) Iteration 31, k=64



(b) Iteration 31, k=256

Fig. 3: KV Cache similarity between layer of 2 different top- k value of Llama-3-8B-Instruct-1048k model.

and thereby creating a critical memory bound bottleneck in autoregressive decoding. In Llama-3-8B-Instruct-Gradient-1048k[11], a 100,000-token context results in a 50GB key-value cache, surpassing the model’s 15.6GB parameter size by over threefold. This substantial memory footprint saturates GPU memory bandwidth, thereby constraining decoding throughput as characterized by the roofline model[23].

To characterize KV Cache sparsity in long-context scenarios, we analyze the 2WikimQA subset of the LongBench benchmark[8], a multi-document QA dataset curated from Wikipedia. From its 200 original samples, we select inputs exceeding 10k tokens to isolate long-context behavior. Our empirical study reveals two key sparsity patterns:

Motivation #1: KV Cache similarity persists across consecutive iterations and adjacent layers.

- **KV Cache similarity between iterations:** Each token generated during the decoding phase is referred to as an iteration. For layer i and iteration j , we extract the top- k attention weights. KV Cache Similarity between iterations is defined as follows:

$$\text{Sim}_{\text{iter}}(i, j, \Delta_j) = \frac{|S_j^{(i)} \cap S_{j+\Delta_j}^{(i)}|}{k} \quad (3)$$

In Equation 3, $S_j^{(i)}$ denotes the set of token indexes for the layer i and the first k attentional weights in the iteration, Δ_j represents the iteration index difference, i represents the depth of the layer, j represents the iteration index, and k represents the top- k value.

As shown in Fig.2, by comparing the KV Cache Similarity between the iteration j and subsequent iterations ($j+1$, $j+2$, and $j+3$), we observe that the similarity between neighboring iterations (e.g. j and $j+1$) is significantly higher than that between non-consecutive iterations (e.g., j and $j+2$, or j and $j+3$). Specifically, when top- k values are set to [64, 128, 256, 512], the average similarity between neighboring iterations reaches 81%-85%, with a peak of 98%. In contrast, KV Cache similarity drops significantly across non-consecutive iterations, highlighting the potential of reusing cached states between adjacent steps to reduce redundancy.

- **KV Cache similarity between layers:** Each token is generated by forwarding through all layers. Similar to

inter-iteration similarity, we measure KV Cache similarity between layer i and subsequent layers ($i+1$, $i+2$, $i+3$). KV Cache Similarity between layers is defined as follows:

$$\text{Sim}_{\text{layer}}(i, j, \Delta_l) = \frac{|S_j^{(i)} \cap S_j^{(i+\Delta_l)}|}{k} \quad (4)$$

As shown in Fig.3, the average KV Cache similarity between layer i and $i+1$ is 93% for $k=64$, and 91% for layer i and $i+3$; for $k=256$, these averages are 73% and 69% respectively. The KV Cache similarity between non-adjacent layers is slightly reduced compared to that between adjacent layers. We observe that increasing top- k reduces inter-layer KV Cache similarity, suggesting that smaller top- k values are preferable for effective cache reuse across layers.

Motivation #2: Nonmonotonic Sparsity Transition in KV Cache Across Layers. Experimental results show that Transformer-based LLMs exhibit significant layer-wise variation in KV Cache sparsity, quantified by the following sparsity index $S_l(\gamma)$:

$$S_l(\gamma) = 1 - \frac{K_l(\gamma)}{N} \quad (5)$$

where $S_l(\gamma)$ denotes the sparsity index of layer_l , $K_l(\gamma)$ denotes the minimum number of KV Cache required to capture the γ attention score in layer_l , and N denotes the total length of the input sequence.

We analyze the relationship between $S_l(96\%)$ and layer depth on the three models Qwen2.5-7B-Instruct-1M[21], Llama-3-8B-Instruct-Gradient-1048k[11] and ChatGLM-4-9B-1M[22]. As shown in Fig.4, we observe that the KV $S_l(96\%)$ evolve with layer depth following a characteristic long-tailed wavelike curve. This phenomenon can be systematically categorized into four distinct phases:

- **Shallow Layers (Layer_{0-4}):** In the initial layers, self-attention focuses on local feature extraction, leading to rapidly increasing sparsity and a reduced KV Cache size needed to capture 96% of attention scores.
- **Intermediate Layers (Layer_{4-10}):** After initial feature aggregation, the model captures broader dependencies, causing a decrease in sparsity and requiring a larger KV Cache to maintain 96% attention coverage.

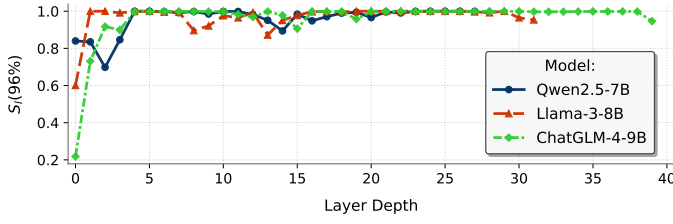


Fig. 4: KV Cache size required to capture 96% attention score for 3 different models with different layers

- **Middle Layers ($Layer_{10-20}$):** These layers integrate global context and encode long-range dependencies, yielding concentrated attention and minimal sparsity, thus requiring a relatively large KV Cache.
- **Deep Layers ($Layer_{20+}$):** At this stage, the model performs high-level semantic synthesis, relying on few key tokens, which significantly increases sparsity and sharply reduces KV Cache size.

An analysis of Llama-3-8B-Instruct-Gradient-1048k[11] reveals a distinct pattern in KV cache Sparsity Index across its layers. Specifically, as shown in Fig 4, in $layer_0$, approximately 39.9% of the KV cache is required to capture 96% of the attention scores. By the $layer_{10}$, this requirement decreases to 2.3%. At the $layer_{13}$ and $layer_{28}$, only about 12.7% and 0.8% of the KV cache are necessary, respectively, to achieve the same coverage. This progression exemplifies a long-tailed wavelike pattern in sparsity evolution.

This observed trend aligns with the hierarchical processing mechanism inherent in Transformer architectures. In the initial layers, the model retains a larger number of tokens to capture various combinations of local characteristics. As processing advances through the intermediate layers, there is a dynamic adjustment of the KV cache size to balance feature extraction with contextual integration. In the deeper layers, the model focuses on efficiently distilling the most representative key information subsets. This long-tail wave-like sparsity pattern suggests that implementing more aggressive KV cache sparsification strategies in the tail-end stages could significantly enhance inference efficiency and overall model performance.

V. DESIGN

A. System overview

In this section, we focus on the details of the SmartCache system design, as shown in Fig.5. We designed a system SmartCache that alleviates access bandwidth bottlenecks and reduces inference latency by selecting and reusing important $TopK$ KV Cache pairs in the decoding phase. Its biggest innovation lies in the rational KV Cache reuse by utilizing the similarity between model layers and iterations, which significantly reduces the overhead of selecting KV Cache, and at the same time effectively reduces the loss of precision and ensures the accuracy of inference.

B. KV Cache Selector

Leveraging KV cache similarity, SmartCache introduces a dual-path selection mechanism that exploits inter-layer and inter-iteration attention pattern correlations. During autoregressive decoding in Transformer-based LLMs, SmartCache dynamically reuses cached states from adjacent layers and prior iterations through a similarity-aware sparse selection algorithm, eliminating redundant recomputation. This approach selectively prioritizes critical tokens from sink and window regions while preserving semantic continuity across decoding steps.

We formulate this reusing process through a similarity-aware sparse selection algorithm, which operates in two dimensions: inter-layer and inter-iteration. The implementation of the algorithm is shown in Algorithm.1. Let $i \in \mathbb{N}^+$ denote the current decoding iteration index, and $j \in \{1, \dots, L\}$ represent the layer index within a transformer model of depth L . We already know that the use of sink-window attention also becomes crucial to ensure high accuracy when selection from the previous iteration or previous layer is required. This is due to the fact that the attention sink (a number of KV Cache at the very beginning position) and the window cache (a number of KV Cache close to the current query) have the most significant impact on the attention score in long-context QA tasks[13]. The selection policy is governed by the following equations:

- **Inter-iteration selection** (from iteration $i - 1$, layer j):
 - Sink region:

$$TopK(\mathcal{K}_{i-1,j}) \cdot (\alpha \cdot \beta) \quad (6)$$

- Window region:

$$TopK(\mathcal{K}_{i-1,j}) (\alpha \cdot (1 - \beta)) \quad (7)$$

- **Inter-layer selection** (from iteration i , layer $j - 1$):
 - Window region:

$$TopK(\mathcal{K}_{i,j-1})(1 - \alpha) \quad (8)$$

As illustrated in Fig. 5, the dataflow architecture exhibits a hierarchical right-angled triangular shaped pattern that systematically decouples spatial and temporal dependencies. This structural design embodies two orthogonal computation paradigms: (1) **Horizontal Context Propagation:** Inter-layer communication maintains spatial coherence by progressively refining local-to-global contextual representations. (2) **Vertical Temporal Capture:** Inter-iteration communication preserves temporal continuity through cross-step state transfer. This vertical pathway enables efficient reuse of historical attention patterns while preventing feature drift across decoding steps.

The inter-iteration similarity factor is denoted by $\alpha \in [0, 1]$, which quantifies the correlation of attention patterns between consecutive decoding iterations. By performing an offline grid-based vectors search on it, we determined that a higher accuracy can be achieved when the value of α is above 0.8, which suggests that SmartCache reuse of the KV Cache from the previous iteration has a higher weight in order to achieve a higher inference accuracy. This can be attributed to temporal

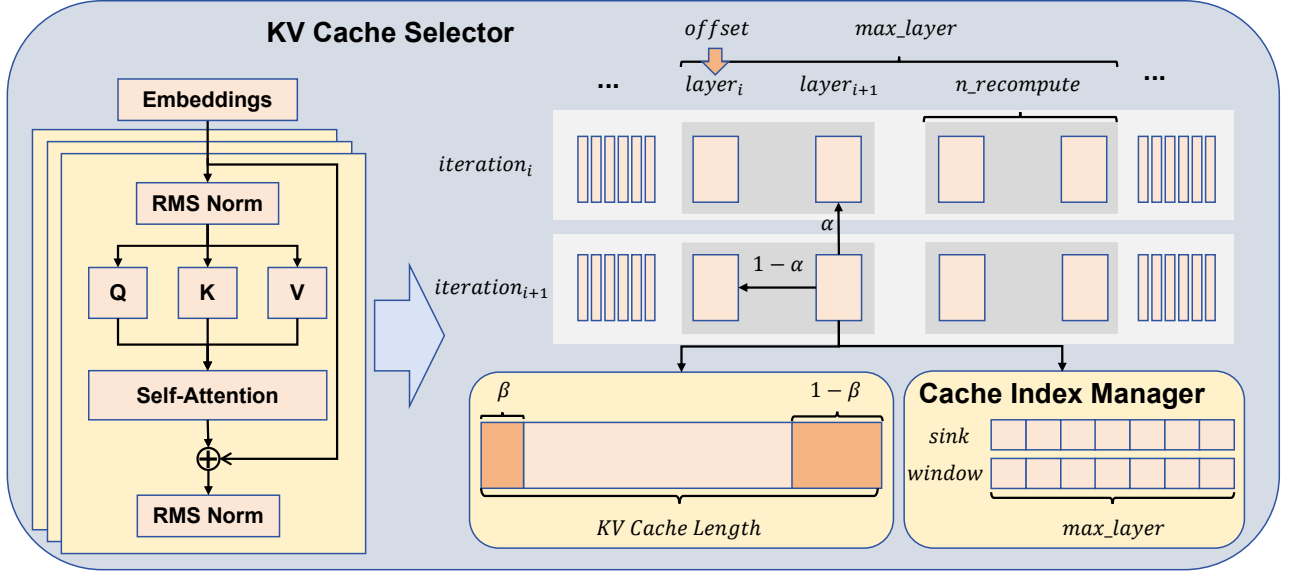


Fig. 5: SmartCache overview.

Algorithm 1 SmartCache KV Cache Sparsity Algorithm

Input $\alpha \in [0, 1]$, $\beta \in [0, 1]$, topk , $n_{\text{recompute}}$, offset , max_layer

Output Selected KV Cache set $\mathcal{K}_{i,j}$ and attention output

```

1: Initialize:  $\mathcal{K}_{\text{inter-iter}} \leftarrow \emptyset$ ,  $\mathcal{K}_{\text{inter-layer}} \leftarrow \emptyset$ 
2: if  $(j \geq \text{offset}) \wedge (j \leq \text{offset} + \text{max\_layer})$  then
3:   if  $j \bmod n_{\text{recompute}} \neq 0$  then
4:      $\mathcal{S}_{\text{sink}} \leftarrow \text{TopK}(\alpha \cdot \beta \cdot \text{topk}, \mathcal{K}_{i-1,j}^{\text{sink}})$ 
5:      $\mathcal{W}_{\text{window}} \leftarrow \text{TopK}(\alpha \cdot (1 - \beta) \cdot \text{topk}, \mathcal{K}_{i-1,j}^{\text{window}})$ 
6:      $\mathcal{K}_{\text{inter-iter}} \leftarrow \mathcal{S}_{\text{sink}} \cup \mathcal{W}_{\text{window}}$ 
7:      $\mathcal{W}'_{\text{window}} \leftarrow \text{TopK}((1 - \alpha) \cdot \text{topk}, \mathcal{K}_{i,j-1}^{\text{window}})$ 
8:      $\mathcal{K}_{\text{inter-layer}} \leftarrow \mathcal{W}'_{\text{window}}$ 
9:      $\mathcal{K}_{i,j} \leftarrow \mathcal{K}_{\text{inter-iter}} \cup \mathcal{K}_{\text{inter-layer}}$ 
10:  else
11:     $\mathcal{K}_{i,j} \leftarrow \bigcup_{m=0}^{n-1} \mathcal{K}_{i,j}^{(m)}$ 
12:  end if
13: else
14:    $\mathcal{K}_{i,j} \leftarrow \mathcal{K}_{\text{full}}$ 
15: end if
16:  $\mathbf{Q}_{i,j} \leftarrow \text{ComputeQuery}(X_{i,j})$ 
17:  $\mathbf{K}_{i,j}, \mathbf{V}_{i,j} \leftarrow \mathcal{K}_{i,j}$ 
18:  $\text{Attention}_{i,j} \leftarrow \text{Softmax}\left(\frac{\mathbf{Q}_{i,j} \mathbf{K}_{i,j}^\top}{\sqrt{d}}\right) \mathbf{V}_{i,j}$ 
19: return  $\text{Attention}_{i,j}, \mathbf{K}_{i,j}, \mathbf{V}_{i,j}$ 

```

continuity of semantic coherence inherent in Transformer-based autoregressive models. Higher α values enforce stronger constraints on attention pattern variations across iterations, effectively preserving historical contextual dependencies. In autoregressive decoding scenarios, adjacent tokens inherently exhibit semantic continuity, and excessive deviation of atten-

tion patterns may disrupt the logical consistency of generated sequences. The KV Cache retention mechanism with $\alpha > 0.8$ establishes a dampening effect on attention divergence.

The sink region preference ratio $\beta \in [0, 1]$ governs the proportion of tokens preferentially selected from the static sink region versus the dynamic window region during KV Cache selection. In each iteration_i and layer_j , SmartCache rigorously partitions the KV cache along two orthogonal dimensions, iteration and layer, by allocating a fraction α of tokens from the previous iteration cache and the complementary fraction $1 - \alpha$ from the cache of the preceding layer. Within the last iteration subsets, it further discriminates between sink and window regions, selecting a proportion β of tokens from the sink and $1 - \beta$ from the window. The sink tokens are entirely sourced from the previous iteration (iteration_{i-1}) because both the previous and current iterations (iteration_i) utilize the identical foundational KV cache state at the current layer (layer_j). Consequently, their initial sink tokens are exactly the same, exhibiting perfect similarity.

As shown in (9), the combined KV Cache set $\mathcal{K}_{i,j}$ is defined through a hierarchical union operation:

$$\mathcal{K}_{i,j} = \underbrace{\mathcal{S}_{i-1,j}^{\text{sink}} \cup \mathcal{W}_{i-1,j}^{\text{window}}}_{\text{Inter-iteration selection}} \cup \underbrace{\mathcal{W}_{i,j-1}^{\text{window}}}_{\text{Inter-layer selection}} \quad (9)$$

Motivated by Fig. 4, this hierarchical selection leverages the observation that deeper layers ($\text{offset} \leq j \leq \text{offset} + \text{max_layer}$) exhibit higher sparsity in attention patterns, enabling aggressive pruning while maintaining model accuracy. As shown in Fig. 5, within $[\text{offset}, \text{offset} + \text{max_layer}]$ we use 2D KV Cache similarity for KV Cache thinning; outside of these layers, to ensure the integrity of the

TABLE I: Performance comparison of GemFilter, StreamingLLM and SmartCache across various LLMs

Task	LongBench V1				InfiniteBench			LooGLE*			LongBench V2		
	HotPotQA	Retrieval	SamSum	2wikimQA	L-DiaQA	L-BookSum	L-BookQA	L-DepQA	S-DepQA	L-DepSum	Overall	Easy	Hard
<i>Llama-3-8B</i>													
All KV	35.74	82.01	42.47	25.5	0.055	0.214	0.161	0.756	0.736	0.745	1.0	1.0	1.6
SmartCache(1): 256	34.03	78.17	38.42	22.71	0.040	0.145	0.140	0.743	0.720	0.735	0.8	0.5	1.0
SmartCache(2): 256	35.85	80.12	41.43	24.82	0.055	0.207	0.154	0.748	0.732	0.732	4.6	4.2	4.8
Gemfilter: 1024	8.15	29.25	39.51	9.79	0.055	0.073	0.010	0.702	0.719	0.660	26.4	27.6	25.7
StreamingLLM: 256	30.56	79.33	38.89	20.52	0.030	0.015	0.008	0.749	0.735	0.729	1.0	0.5	1.3
<i>Qwen2.5-7B</i>													
All KV	47.98	95.18	44.75	38.0	0.065	0.127	0.014	0.762	0.737	0.760	30.3	37.0	25.4
SmartCache(1): 256	46.89	89.85	37.53	36.03	0.055	0.097	0.161	0.780	0.759	0.749	24.7	30.5	21.5
SmartCache(2): 256	47.85	95.57	44.8	39.63	0.060	0.214	0.014	0.801	0.783	0.795	30.2	34.8	25.9
Gemfilter: 1024	12.35	67.52	43.88	10.22	0.070	0.111	0.011	0.739	0.850	0.782	28.1	29.4	26.7
StreamingLLM: 256	38.31	52.05	41.07	34.63	0.031	0.014	0.012	0.762	0.739	0.760	29.8	35.9	24.5
<i>glm4-9B</i>													
All KV	15.03	95.17	41.13	15.92	0.055	0.177	0.016	0.706	0.705	0.685	30.8	32.3	30.2
SmartCache(1): 256	14.96	92.79	41.28	16.78	0.045	0.116	0.012	0.716	0.709	0.691	30.1	31.8	29.3
SmartCache(2): 256	15.09	93.67	44.21	16.78	0.070	0.172	0.015	0.710	0.707	0.687	30.6	32.3	30.3
Gemfilter: 1024	8.83	27.19	27.33	10.82	0.050	0.107	0.023	0.702	0.709	0.660	25.3	25.0	27.0
StreamingLLM: 256	11.65	73.83	39.88	13.36	0.100	0.161	0.017	0.712	0.703	0.686	29.8	30.2	30.2

*Note: In LooGLE benchmark we choose bertscore as the accuracy metric.

information in the inference process, we use the full KV Cache for inference.

To mitigate potential information degradation caused by prolonged KV Cache reuse, SmartCache incorporates a periodic recomputation strategy that strategically refreshes the KV Cache through forward passes. This mechanism is governed by $n_recompute$ hyperparameter, which serves as the recomputation interval to balance computational overhead and accuracy. Specifically, when $layer_idx$ satisfies: $layer_idx \bmod n_recompute = 0$, SmartCache discards accumulated KV Cache and regenerates it through complete layer-wise recomputation, thereby eliminating error propagation from recursive state updates. The recomputation strategy we designed is shown in (10):

$$\text{Recompute}(j) = \begin{cases} \text{True} & \text{if } j \bmod n_recompute = 0 \\ \text{False} & \text{otherwise} \end{cases} \quad (10)$$

When triggered (i.e. $\text{Recompute}(j) = \text{True}$), SmartCache performs full attention computation using the complete KV Cache set:

$$\mathcal{K}_{\text{full}} = \bigcup_{m=0}^{n-1} \mathcal{K}_{i,j}^{(m)} \quad (11)$$

For example, when $n_recompute = 2$, we say that the number of single reuse steps is 2. This ensures that SmartCache uses the full amount of KV cache computation every $n_recompute$ times, ensuring the integrity of the information.

This two-dimensional similarity-aware selection Algorithm ensures that both inter-iteration and inter-layer affinities are systematically exploited to optimize attention computation. The combined KV Cache enables efficient computation

through sparse attention mechanisms. For query $Q_{i,j}$ at iteration i and layer j , the attention scores are computed as:

$$\text{Attention}(Q_{i,j}, \mathcal{K}_{i,j}) = \text{Softmax} \left(\frac{Q_{i,j}(\mathcal{K}_{i,j})^\top}{\sqrt{d_k}} \right) \cdot V_{i,j} \quad (12)$$

In long context scenarios, efficiently selecting part of the KV Cache for attention computation alleviates the access bottleneck in the decoding phase and effectively reduces the inference latency, while the efficient and accurate reuse of SmartCache ensures the accuracy of inference.

C. Cache Index Manager

According to the design of SmartCache LLMs decoding to $layer_j$ of iteration i require the KV Cache from $iteration_{i-1}$ and $layer_{j-1}$. A naive solution would be to additionally write the $TopK$ of $K_{i,j}$ and $V_{i,j}$ to HBM when the system computes them. However, we note that the system can utilize the already saved KV cache and simply store the KV Cache indexes that need to be reused.

According to what is stated in Motivation #1, the KV Cache Similarity between non-adjacent layers and iterations is lower than that of adjacent layers. In order to ensure the accuracy of the inference results, SmartCache set the hyperparameter $n_recompute = 2$, the system only needs to save the KV Cache of the $TopK$ of this layer for the next layer. Thus, when storing indexes, SmartCache uses rolling arrays, requiring only one layer of Cache Index space:

$$M_{\text{naive}} = \mathcal{O}(L \cdot n) \rightarrow M_{\text{ours}} = \mathcal{O}(n) \quad (13)$$

As shown in (13), L stands for max_layer number of transformer layers, n stands for $TopK$. Our designed Cache Index Manager stores the index information of KV Cache with

very low memory footprint, solves the high latency overhead problem of reselecting the scheme before computing attention, and dramatically improves the inference efficiency.

VI. EVALUATION

A. Setup

System Implementation. We developed a complete sparse inference system using Python and Pytorch[30], including all the features of SmartCache, to enable efficient inference for the decoding phase of long context scenarios. To this end, we built an inference engine that includes a KV Cache Selector, a Memory Manager, and support for accelerating inference using optimizations such as Flashattention[12].

Models and Tasks. To verify the effectiveness of the SmartCache system, we chose three widely used long-text models: Llama-3-8B-Instruct-Gradient-1048k[31], Qwen2.5-7B-Instruct-1M[21] and glm-4-9b-chat-1m[22] for our experiments. We chose different types of tasks from four different benchmarks[7–10], including single-document QA, multi-document QA, long in-context learning, long-dialogue history understanding, code understanding, the mathematical and logical computation tasks to test the performance of SmartCache comprehensively and exhaustively.

Baselines. We chose the current widely used KV Cache sparse algorithm StreamingLLM[13], the GemFilter[16] algorithm as the baseline, as a comparison along with the full amount of KV Cache configurations, as well as two configurations of SmartCache StreamingLLM[13] statically reserves KV Cache at sink position and window position at each layer to achieve sparsity; Gemfilter[16] uses the first few layers of the model to select critical KV Cache to be retained prior to the prefill phase, and uses both prefill and decode phases with a small number of critical KV Cache for computation.

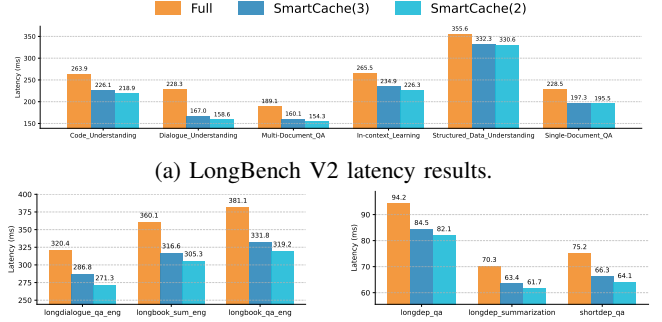
B. Accuracy Results

The results of the accuracy evaluation on the four benchmarks for SmartCache, All KV, GemFilter[16] and StreamingLLM[13] are shown in Table .I. The LooGLE[10] accuracy metrics are judged using BertScore[32], the summary abstract type task is judged using ROUGE-L[33] metrics, the QA type task is judged using F1 score, and the code comprehension task is judged using the fuzzywuzzy library for similarity. The results show that the model inference accuracy degradation is within 2% in most of the test tasks, and more than 1% inference accuracy improvement is obtained in more than half of the tasks, with the highest improvement being 6.24%.

In order to verify the effect of the offset in our KV Cache Selector, we design two SmartCache configurations, in which the configuration offset of SmartCache(2) is deeper, corresponding to the long-tailed wavelike pattern described in the previous section. According to the conclusion in Motivation #2, the inter-layer similarity of KV Cache decreases more obviously when the *topk* value is larger, so we choose $\alpha = 0.8$, $\beta = 0.2$. In order to trade off accuracy and delay, we choose

TABLE II: SmartCache configuration details

Config	α	β	offset	max_layer	n_recompute
SmartCache(1)	0.8	0.2	12	12	2
SmartCache(2)	0.8	0.2	16	12	2
SmartCache(3)	0.8	0.2	16	8	2



(b) InfiniteBench latency results. (c) LooGLE latency results.

Fig. 6: Latency results of Llama-3-8B-Instruct-Gradient-1048k

$n_recompute = 2$, $max_layer = 12$ as the configuration of accuracy evaluation. The two configurations selected for SmartCache are shown in Table.II.

In the results of LongBench V1, **SmartCache achieved better accuracy than All KV Cache in all four tasks of HotPotQA, Retrieval, SamSum and 2WikimQA**. This is due to the fact that the SmartCache system removes long by exploiting the two-dimensional similarity of the KV Cache for sparse toxic information in the text.

In the results of LongBench V1, SmartCache achieves better accuracy than All KV Cache in all four tasks of HotPotQA, Retrieval, SamSum and 2WikimQA, due to the fact that the SmartCache system removes long toxic information in the text. According to the test results in Table.I, Llama-3-8B-Instruct-Gradient-1048k, Qwen2.5-7B-Instruct-1M and glm-4-9b-chat-1m accuracies on LongBench V1 have maximum accuracy degradation of 2.67%, 0.27% and 1.57% respectively, and more than half of the task precision does not decrease but increases, reflecting the feature of SmartCache precision almost no loss. For GemFilter and StreamingLLM, the precision reduction can be up to 77.19% and 45.31%. Although they also have performance with accuracy reduction within 10% (e.g., on the SamSun task), their accuracy and stability are not as good as that of SmartCache

InfiniteBench contains 100k+ of very long text ranging across the diverse domains of real-world data. SmartCache similarly achieves better accuracy than KV Cache in all three tasks, L-DiaQA, L-BookSum and L-BookQA. Llama-3-8B-Instruct-Gradient-1048k, Qwen2.5-7B-Instruct-1M and glm-4-9b-chat-1m accuracies have a maximum accuracy degradation of 4.34%, 7.69%, and 6.25% on InfiniteBench, respectively, but can achieve a maximum of 68.5% accuracy improvement. This may be related to the unfavourable fine-tuning of the model during length extrapolation, which we do not focus on.

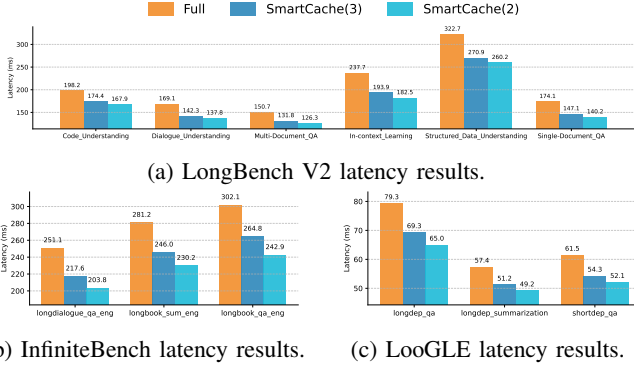


Fig. 7: Latency results of Qwen2.5-7B-Instruct-1M

While only the Llama-3-8B-Instruct-Gradient-1048k model achieved 0.54%-1.34% accuracy degradation in the LooGLE accuracy test results, the other two test models both achieved 0.87%-6.24% accuracy improvement, which indicates that the SmartCache system is fully capable of sparse long text when it is ability to guarantee the inference accuracy of All KV Cache. For the test results of LongBench V2, Llama-3-8B-Instruct-Gradient-1048k we do not analyse the results due to the model capability. The accuracy reduction ranges from 0.33% to 5.94% on the Qwen2.5-7B-Instruct-1M and glm-4-9b-chat-1m models, and an accuracy improvement of about 1% is always obtained on the hard task.

C. Latency Results

In order to demonstrate the decoding acceleration effect of SmartCache on the ultra-long sequence task, we firstly show the latency results of the Llama-3-8B-Instruct-Gradient-1048k model tested on three longer sequence benchmarks, as shown in Fig.6.

The comparison of SmartCache(2) and SmartCache(3) demonstrates the impact of max_layer on latency. On the Dialogue understanding task SmartCache system compared to Full KV Cache on average can obtain 30.5% decode latency gain, this is due to the average prompt length of more than 100k in the Dialogue understanding task, SmartCache SmartCache system is more effective in accelerating very long sequences. The average latency gain on LongBench V2, InfiniteBench and LooGLE is 17.03%, 15.6% and 13.2% respectively.

As illustrated in Fig.7, the Qwen2.5-7B-Instruct-1M model reduced average latency by 23.2 % in long in-context learning tasks encompassing user-guide QA, novel language translation and n-shot learning compared with a full KV cache. Fig.8 shows that glm-4-9b-chat-1m model achieved a 23.6 % delay reduction in long dialogue history understanding tasks covering both agent and dialogue QA. Under SmartCache(2) configuration Qwen2.5-7B-Instruct-1M saw a 23.6 % reduction on LongBench V2 and further gains of 18.7 %, 18.9 % and 15.8 % on InfiniteBench and LooGLE. The glm-4-9b-chat-1m model realized reductions of 17.5 %, 12.3 % and 12.3 % on those same benchmarks. Its deeper 40-layer architecture accounts for its comparatively longer decoding time.

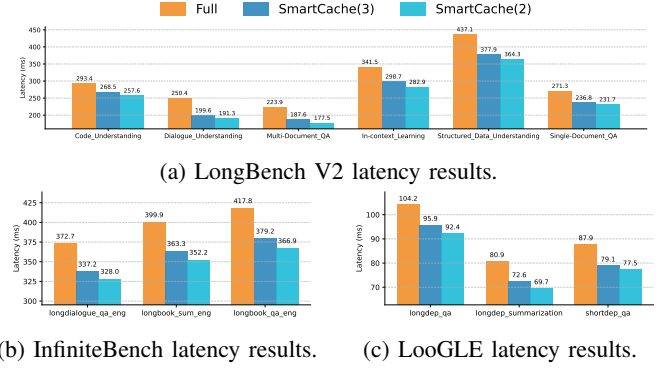


Fig. 8: Latency results of glm-4-9b-chat-1m

VII. CONCLUSION

We have presented SmartCache, a novel inference system that exploits the two-dimensional similarity of KV cache to enable efficient, high-accuracy sparsification in long-context LLM decoding. By combining a dual path selection algorithm that is aware of similarity with a rolling array cache index manager, SmartCache dynamically reuses only the most critical key-value pairs while maintaining semantic consistency, reducing computational and memory-access overhead. Extensive experiments on Llama-3-8B-Instruct-Gradient-1048k, Qwen2.5-7B-Instruct-1M, and glm-4-9b-chat-1m across four long-context benchmarks demonstrate that SmartCache achieves up to 30.5% end-to-end decoding latency reduction and average latency improvements of 15%–23% across diverse scenarios, with inference accuracy degradation consistently within 2% and in many cases even showing slight improvements.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, and I. Akkaya, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, and A. Gulati, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” *arXiv preprint arXiv:2403.05530*, 2024.
- [3] A. Anthropic, “The claude 3 model family: Opus, sonnet, haiku,” *Claude-3 Model Card*, vol. 1, p. 1, 2024.
- [4] H. Liu, W. Yan, M. Zaharia, and P. Abbeel, “World model on million-length video and language with blockwise ringattention,” *arXiv preprint arXiv:2402.08268*, 2024.
- [5] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “Yarn: Efficient context window extension of large language models,” *arXiv preprint arXiv:2309.00071*, 2023.
- [6] S. Tworowski, K. Staniszewski, M. Pacek, Y. Wu, H. Michalewski, and P. Miłoś, “Focused transformer: Contrastive training for context scaling,” *Advances in neural information processing systems*, vol. 36, pp. 42 661–42 688, 2023.

- [7] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, and X. Lv, "Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," *arXiv preprint arXiv:2412.15204*, 2024.
- [8] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, and Z. Huang, "Longbench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, aug 2024, pp. 3119–3137. [Online]. Available: <https://aclanthology.org/2024.acl-long.172>
- [9] X. Zhang, Y. Chen, S. Hu, Z. Xu, J. Chen, and M. Hao, " ∞ Bench: Extending long context evaluation beyond 100K tokens," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, aug 2024, pp. 15 262–15 277. [Online]. Available: <https://aclanthology.org/2024.acl-long.814>
- [10] J. Li, M. Wang, Z. Zheng, and M. Zhang, "Loogle: Can long-context language models understand long contexts?" *arXiv preprint arXiv:2311.04939*, 2023.
- [11] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, and A. Al-Dahle, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [12] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.
- [13] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023.
- [14] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, and H. Ye, "Snapkv: Llm knows what you are looking for before generation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 22 947–22 970, 2024.
- [15] Z. Cai, Y. Zhang, B. Gao, Y. Liu, T. Liu, and K. Lu, "Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling," *arXiv preprint arXiv:2406.02069*, 2024.
- [16] Z. Shi, Y. Ming, X.-P. Nguyen, Y. Liang, and S. Joty, "Discovering the gems in early layers: Accelerating long-context llms with 1000x input token reduction," *arXiv preprint arXiv:2409.17422*, 2024.
- [17] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference, 2024," URL <https://arxiv.org/abs/2406.10774>.
- [18] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, and S. Ahn, "Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention," *Advances in Neural Information Processing Systems*, vol. 37, pp. 52 481–52 515, 2024.
- [19] Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, and N. Nolte, "Magicpig: Lsh sampling for efficient llm generation," *arXiv preprint arXiv:2410.16179*, 2024.
- [20] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, and Z. Zhang, "Inflm: Training-free long-context extrapolation for llms with an efficient context memory," *arXiv preprint arXiv:2402.04617*, 2024.
- [21] Q. Team, "Qwen2.5-1m: Deploy your own qwen with context length up to 1m tokens," January 2025. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-1m/>
- [22] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, and D. Yin, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," 2024.
- [23] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, and C. Xue, "Llm inference unveiled: Survey and roofline model insights," *arXiv preprint arXiv:2402.16363*, 2024.
- [24] S. Imai, R. Nakazawa, M. Amaral, S. Choochotkaew, and T. Chiba, "Predicting llm inference latency: A roofline-driven ml method," in *Annual Conference on Neural Information Processing Systems*, 2024.
- [25] S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, and H. Genc, "Full stack optimization of transformer inference: a survey," *arXiv preprint arXiv:2302.14017*, 2023.
- [26] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, and R. Cai, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.
- [27] W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. Ragan-Kelley, "Reducing transformer key-value cache size with cross-layer attention," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [28] M. Adnan, A. Arunkumar, G. Jain, P. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.
- [29] H. Sun, L.-W. Chang, W. Bao, S. Zheng, N. Zheng, and X. Liu, "Shadowkv: Kv cache in shadows for high-throughput long-context llm inference," *arXiv preprint arXiv:2410.21465*, 2024.
- [30] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, and G. Chanan, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.
- [31] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [32] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with bert," *arXiv preprint arXiv:1904.09675*, 2019.
- [33] K. Ganesan, "Rouge 2.0: Updated and improved measures for evaluation of summarization tasks," *arXiv preprint arXiv:1803.01937*, 2018.